

" OrbitMind AI– Intelligent Productivity & Study Companion"

by

Md. Imroz Newaz

CSE 11220321052

Zeniya Naznin

CSE 11230121138

Fatema Parvin Kanta

CSE 11230121192

Md. Mahfuzur Rahman Hridoy

CSE 11230121186

WEEK 06 ASSIGNMENT

Course Title: Software Development III: AI-Based Software Project Development

Course Code: CSE 4104

Section: 7D | Spring 2026



Submitted to

Md. Riaz Mahmud

Assistant Professor

Department of Computer Science and Engineering

Northern University of Business and Technology, Khulna

Khulna 9100, Bangladesh

CONTENTS

Title Page	1
Contents	2
List of Figures	3
CHAPTER I – Team Information	4
1.1 Team Information Sheet	4
1.1.1 Team Members	4
CHAPTER II – Backend Architecture & Implementation	4-7
2.1 Backend Technology Stack & Architecture	4-5
2.2 Database Implementation & Schema Design	5
2.3 Authentication System Workflow	5-6
2.4 Error Handling & Validation Architecture	6
2.5 Core AI Development Progress	6-7
2.6 Current Development Progress	7
CHAPTER III – Visual Evidence & API Testing	8-18
3.1 Backend Folder Structure	8
3.2 Database Tables / Collections	9-13
3.2.1 User Collection	9-10
3.2.2 Tasks Collection	10-11
3.2.3 Study Plan Collection	11
3.2.4 Chat Logs Collection	12
3.2.5 Summaries Collection	12-13
3.2.6 Notifications Collection	13
3.3 Authentication Process (Code)	14
3.4 API Testing Results (Postman)	14-16
3.5 GitHub Repository	17-18
3.5.1 GitHub Repository Link	18

List of Figures

3.1	Figure: Organized MVC Backend Directory Structure	8
3.2	Figure: Overview of all Initialized Collections	9
3.2.1	Figure: The Users Collection demonstrating secure data storage.	10
3.2.2	Figure: The Tasks Collection highlighting relational mapping	10
3.2.3	Figure: The StudyPlans Collection storing AI-generated data	11
3.2.4	Figure: The ChatLogs Collection for AI Coach history.	12
3.2.5	Figure: The Summaries Collection for condensed lecture notes.	12
3.2.6	Figure: The Notifications Collection for system alerts.	13
3.3	Figure: Secure password Hashing & JWT generation logic	14
3.4.1	Figure: Successful API Testing of the Registration Endpoint	15
3.4.2	Figure: Successful API Testing of the Login Endpoint	15
3.4.3	Figure: Failure API Testing of the Generate Endpoint	15
3.4.4	Figure: Successful API Testing of the Task Endpoint	16
3.5	Figure: Maintained GitHub Repository	17

CHAPTER I

1.1 Team Information Sheet

Field	Details
Team Name	CSE4104-7D-T04
Section	7D
Project Title	OrbitMind AI — Smart Productivity & Study Assistant
Team Leader	Md. Imroz Newaz (ID: 11220321052)

1.1.1 Team Members

Member Name	Student ID	Role
Md. Imroz Newaz	11220321052	Team Leader & Database Manager
Zeniya Naznin	11230121138	Frontend Developer
Fatema Parvin Kanta	11230121192	Backend Developer
Md. Mahfuzur Rahman Hridoy	11230121186	AI Integration Lead

CHAPTER II

2.1 Backend Technology Stack & Architecture

Our team has successfully established the foundational backend architecture utilizing a modern, scalable technology stack.

- **Runtime Environment:** Node.js (Chosen over Django/Laravel because its asynchronous, event-driven nature is perfectly suited for handling concurrent API requests to the Google Gemini LLM without blocking the main thread).

- **Web Framework:** Express.js (Selected for its robust, modular routing and lightweight middleware capabilities).
- **Database:** MongoDB Atlas (A NoSQL cloud database selected for its flexible JSON-based document structure, which is critical for storing dynamic, unpredictable AI responses).
- **Security & Authentication:** *bcryptjs* for secure password hashing and *jsonwebtoken* (JWT) for stateless, highly scalable session management.

2.2 Database Implementation & Schema Design

Based on the ER diagram developed in Week 04, we have successfully implemented the Mongoose schemas and established secure connectivity to our MongoDB Atlas cluster.

- **Users Collection:** Stores *full_name*, *email* (Unique Index), *password_hash*, *university*, and *department*. Enforces strict email regex validation.
- **Tasks Collection:** Stores *title*, *description*, *deadline*, *priority_level*, and *is_completed*.
- **StudyPlans Collection:** Stores the AI-generated *roadmap_schedule* JSON array, *subject_name*, and *target_exam_date*.
- **Relational Integrity in NoSQL:** We utilized Mongoose *ObjectId* references as Foreign Keys. Every *Task* and *StudyPlan* document contains a *user_id* field. This ensures absolute data isolation; students can only retrieve and modify their own academic data.

2.3 Authentication System Workflow

We have implemented a highly secure, stateless authentication system that ensures our API endpoints remain protected from unauthorized access.

- **User Registration:** When a student registers, the backend intercepts the raw password. We use *bcryptjs* (with a salt factor of 10) to hash the password before saving the new document to MongoDB.
- **User Login:** During login, the server queries the database by email. *bcrypt.compare()* is used to securely verify the hashed password against the user's input.

- **Session Management (JWT):** Upon successful verification, the Express server signs a secure JSON Web Token (JWT) using a hidden *JWT_SECRET* stored in our *.env* file. This token contains the user's ID and an expiration limit.
- **Route Protection Middleware:** We developed a custom *authMiddleware.js* function. Any request to a protected route must include the JWT in the *Authorization: Bearer <token>* HTTP header.
- **User Role Management:** We currently enforce a strict default 'Student' role across the platform to ensure streamlined access to the AI features, with architectural scalability in place to support 'Admin' roles in the future.

2.4 Error Handling & Validation Architecture

To ensure the system does not crash under invalid requests, we have implemented comprehensive error handling across all backend routes.

- **Input Validation:** API routes verify that all mandatory fields (e.g., email, password) are present before hitting the database. If missing, the API returns a *400 Bad Request* with a specific error message.
- **Authentication Failures:** If a user enters an incorrect password or an unregistered email, the system safely catches the error and returns a generic *401 Unauthorized* message ("Invalid Credentials") to prevent security reconnaissance.
- **Global Error Catcher:** We implemented an Express global error-handling middleware (*app.use((err, req, res, next) => {...})*). Any unexpected database or API exception is caught here, preventing server crashes and returning a controlled *500 Internal Server Error* response.

2.5 Core API Development Progress

We have successfully routed, connected to the database, and tested the core APIs required for the initial phase of the system.

- **Authentication APIs:**
 - *POST /api/auth/register* → Returns 201 Created & JWT Token.

- ***POST /api/auth/login*** → Returns 200 OK & JWT Token.
- **Task Management APIs (Protected):**
 - ***GET /api/tasks*** → Returns 200 OK & JSON array of user's tasks.
 - ***POST /api/tasks*** → Returns 201 Created & saved task object.
- **AI Integration APIs (Protected):**
 - ***POST /api/studyplans*** → Proxies data to Gemini and returns 201 Created with generated roadmap.
 - ***POST /api/chat*** → Returns 200 OK with AI Coach response.

2.6 Current Development Progress

We are proud to report that the Backend Development and Database Implementation phase is fully operational. The Node.js server successfully communicates with MongoDB Atlas, all major CRUD operations are functioning, and data is persisting perfectly. With our JWT authentication fully secured, robust error handling in place, and our endpoints rigorously tested via Postman, the backend is now completely prepared for frontend UI integration in Week 07.

CHAPTER III

3.1 Backend Folder Structure

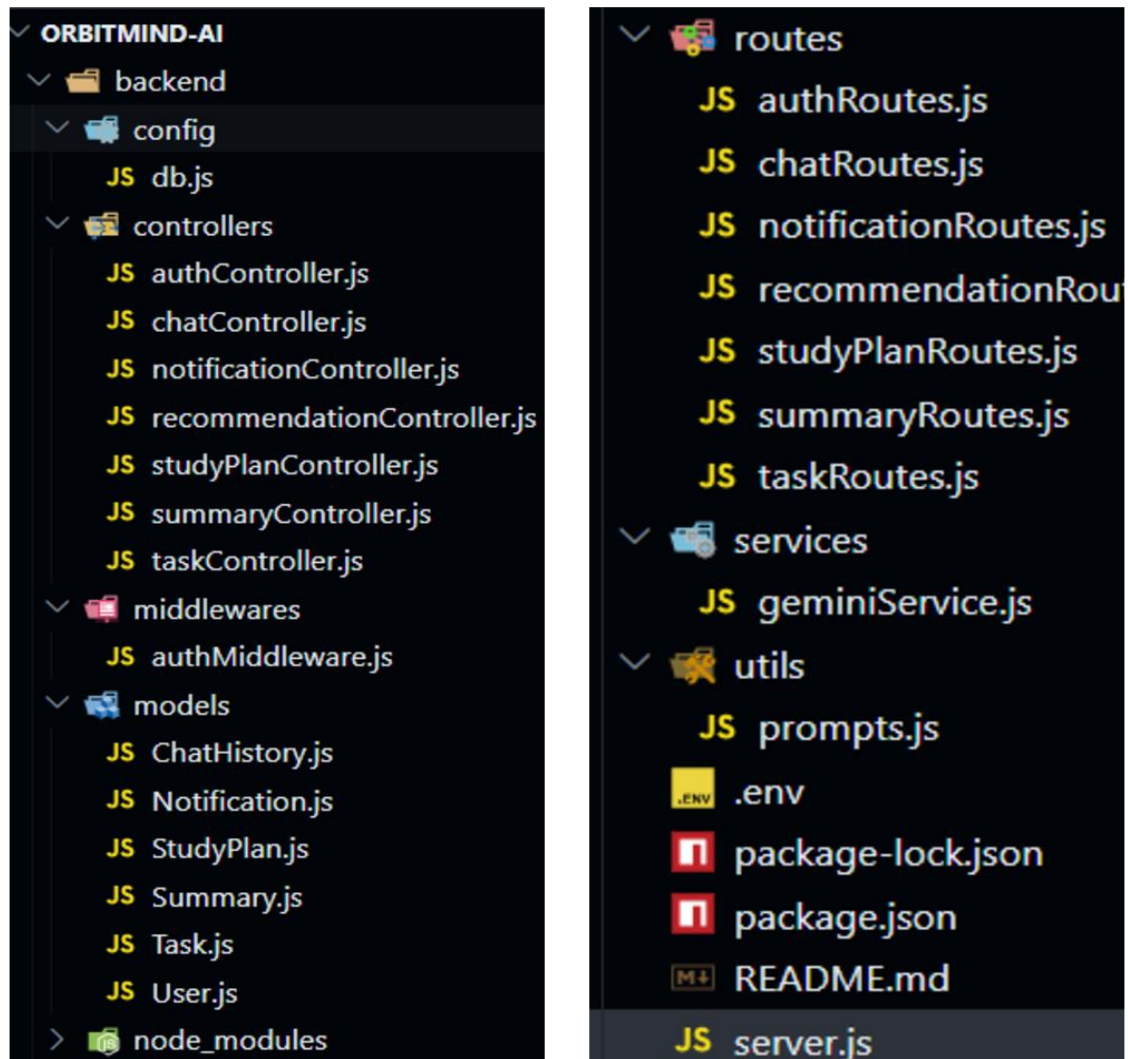


Figure 3.1: Organized MVC Backend Directory Structure in VS Code.

Implementation Details: This screenshot demonstrates our adherence to the MVC (Model-View-Controller) design pattern. By separating routing logic, controller functions, and database models into dedicated directories, we ensure the backend remains scalable, modular, and easy for multiple developers to navigate without causing merge conflicts.

3.2 Database Tables / Collections

Overview

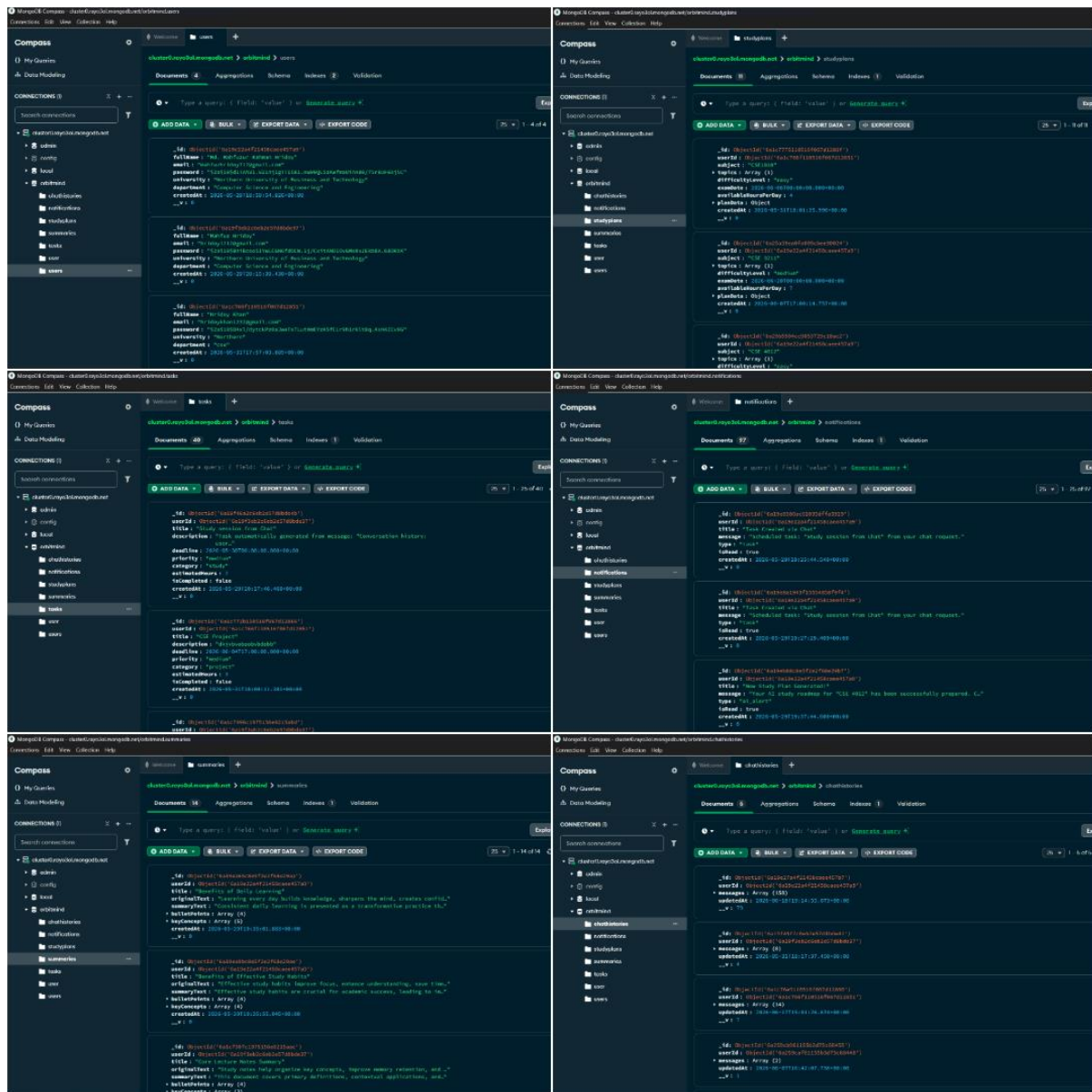


Figure 3.2: Overview of all Initialized Collections in MongoDB Atlas.

Implementation Details: Displayed here is the complete list of active collections securely hosted on our MongoDB Atlas cloud cluster. The successful instantiation of these collections proves our Node.js database connectivity is fully operational and mirrors our ER Diagram.

3.2.1 Users Collection

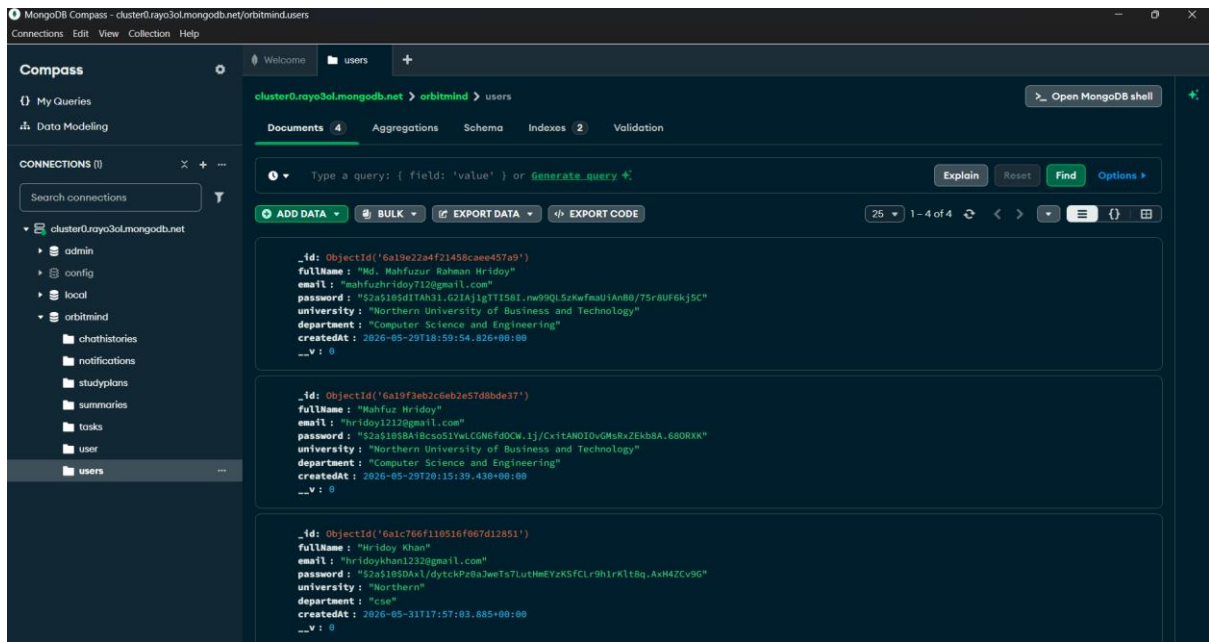


Figure 3.2.1: The Users Collection demonstrating secure data storage.

Implementation Details: This collection stores all registered students. Notice the *password* field is securely hashed via *bcryptjs*, ensuring that sensitive student credentials are never stored in plain text.

3.2.2 Tasks Collection

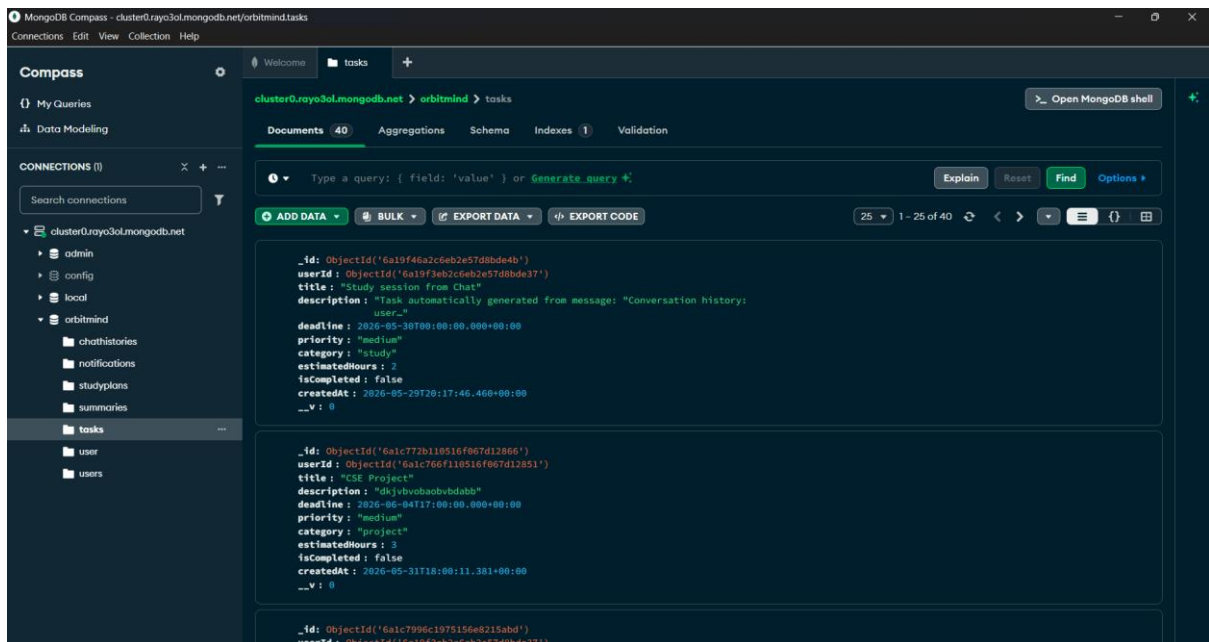


Figure 3.2.2: The Tasks Collection highlighting relational mapping.

Implementation Details: This collection holds the students' academic tasks. Each document utilizes a Mongoose *ObjectId* as a foreign key (*user_id*) to strictly map the task to the correct student, ensuring absolute data isolation.

3.2.3 Study Plans Collection

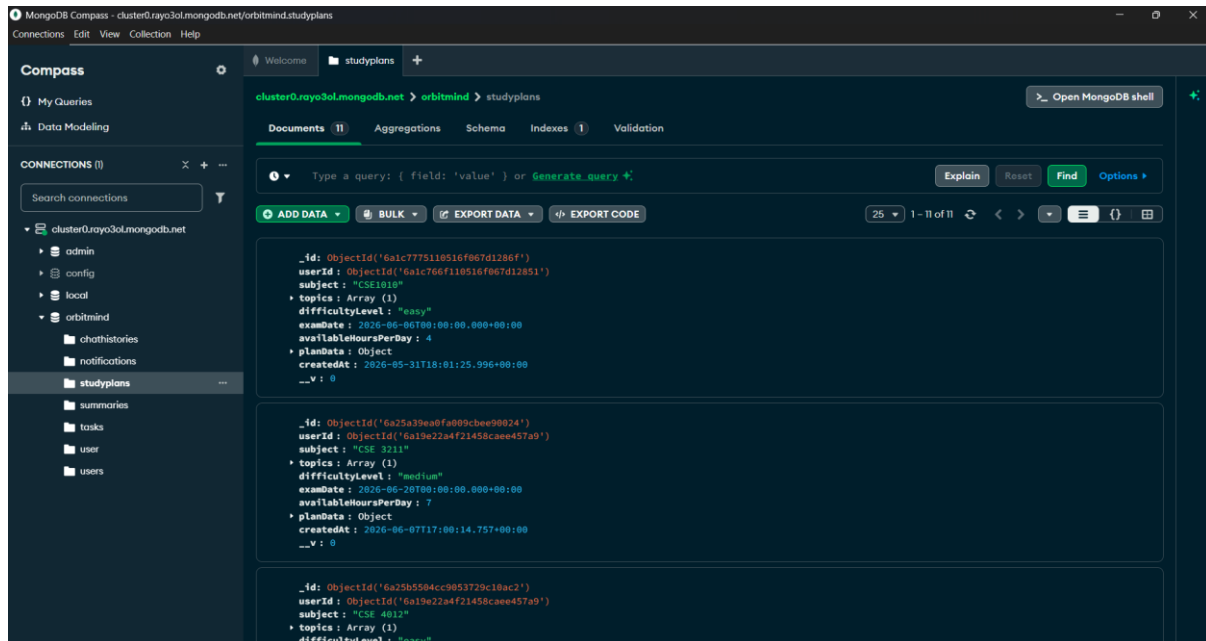


Figure 3.2.3: The StudyPlans Collection storing AI-generated data.

Implementation Details: This collection dynamically stores the complex, JSON-based study roadmaps generated by the Google Gemini AI, proving our NoSQL database's flexibility in handling nested array structures.

3.2.4 Chat Logs Collection

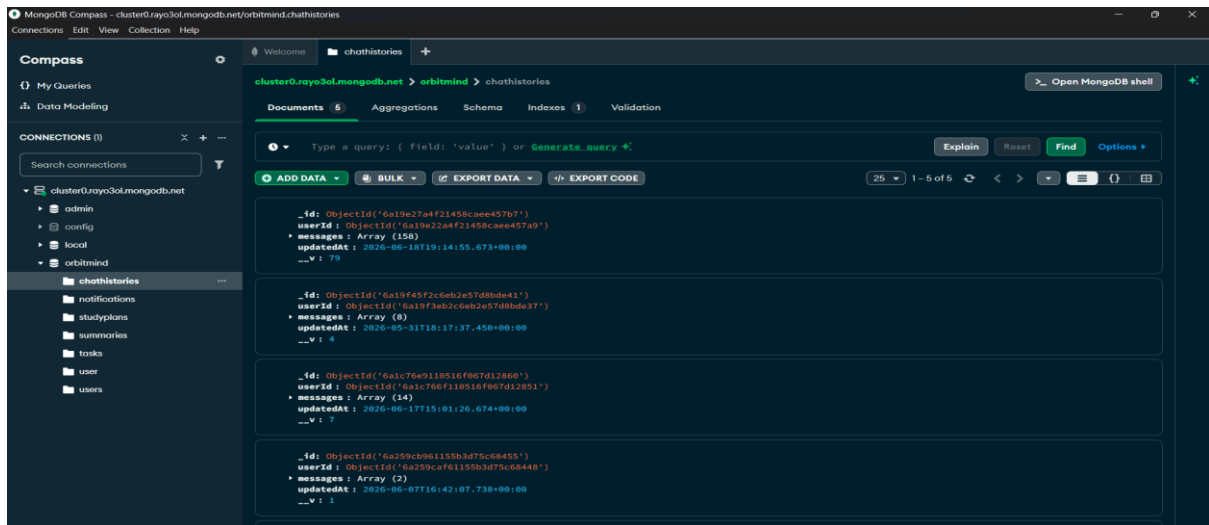


Figure 3.2.4: The ChatLogs Collection for AI Coach history.

Implementation Details: This collection saves the conversational history between the student and the AI Coach, allowing for continuous, context-aware assistance across multiple sessions.

3.2.5 Summaries Collection

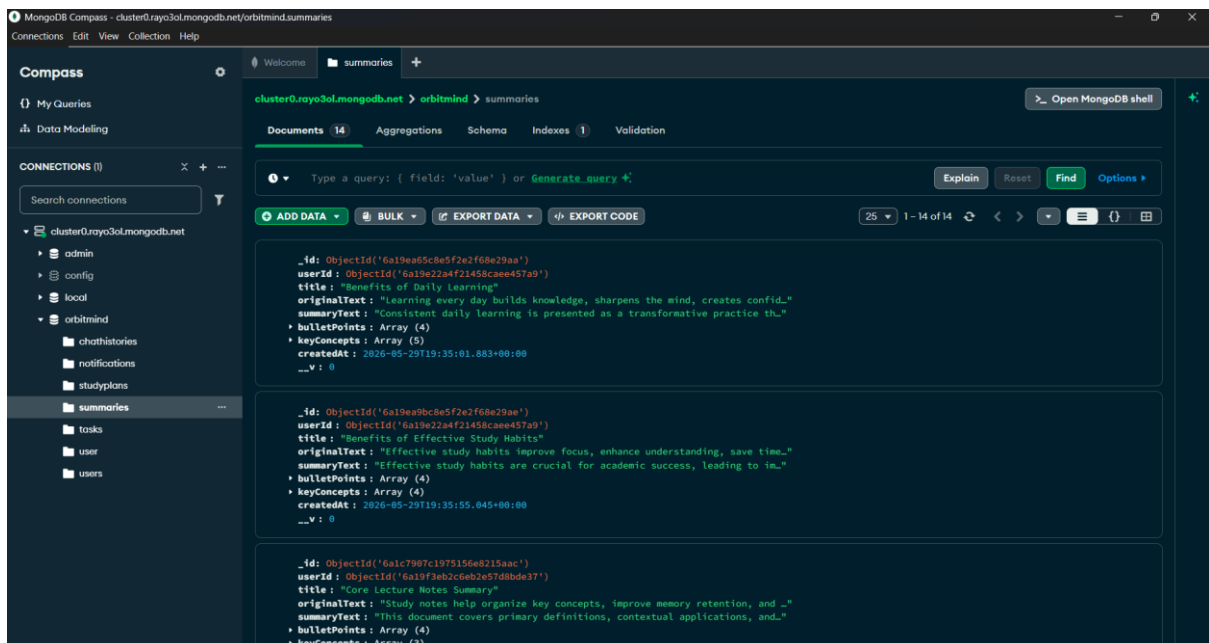


Figure 3.2.5: The Summaries Collection for condensed lecture notes.

Implementation Details: This collection stores the output of the AI Note Summarizer, allowing students to securely retrieve their generated academic summaries at any time without re-prompting the AI.

3.2.6 Notifications Collection

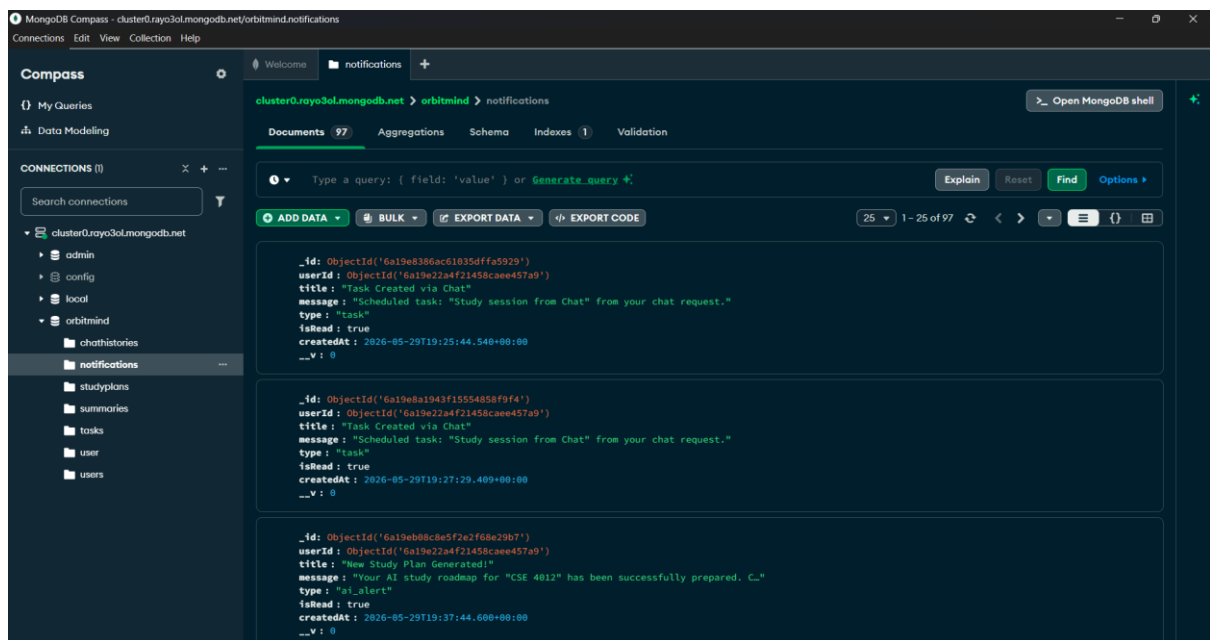


Figure 3.2.6: The Notifications Collection for system alerts.

Implementation Details: This collection manages all system-generated alerts (such as approaching deadlines or AI study reminders). It is designed for high-speed read/write operations to ensure students receive real-time updates on their dashboard.

3.3 Authentication Process (Code)

```
5 const generateToken = (id) => {
6   return jwt.sign({ id }, process.env.JWT_SECRET || 'orbitmind_jwt_fallback_secret_key', {
7     expiresIn: '30d'
8   });
9 };
10
```

```
41
42     const salt = await bcrypt.genSalt(10);
43     const hashedPassword = await bcrypt.hash(password, salt);
44
45
46     const user = await User.create({
47       fullName,
48       email,
49       password: hashedPassword,
50       university,
51       department
52     });
```

Figure 3.3: Secure Password Hashing and JWT generation logic implemented in the Auth Controller.

Implementation Details: This code snippet highlights our advanced authentication architecture. The *bcryptjs* library is utilized to hash passwords before database insertion to prevent plain-text vulnerabilities, while the *jsonwebtoken* library generates a secure, signed payload containing the user's unique ID for stateless session management.

3.4 API Testing Results (Postman)

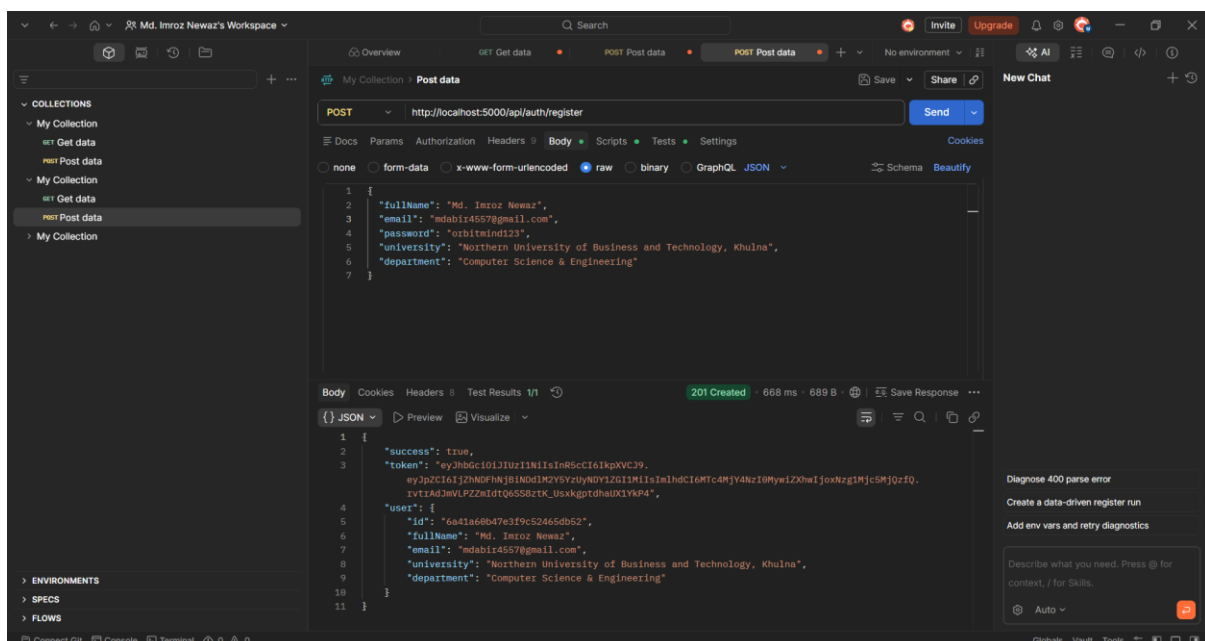


Figure 3.4.1: Successful API Testing of the Registration Endpoint using Postman, demonstrating database connectivity.

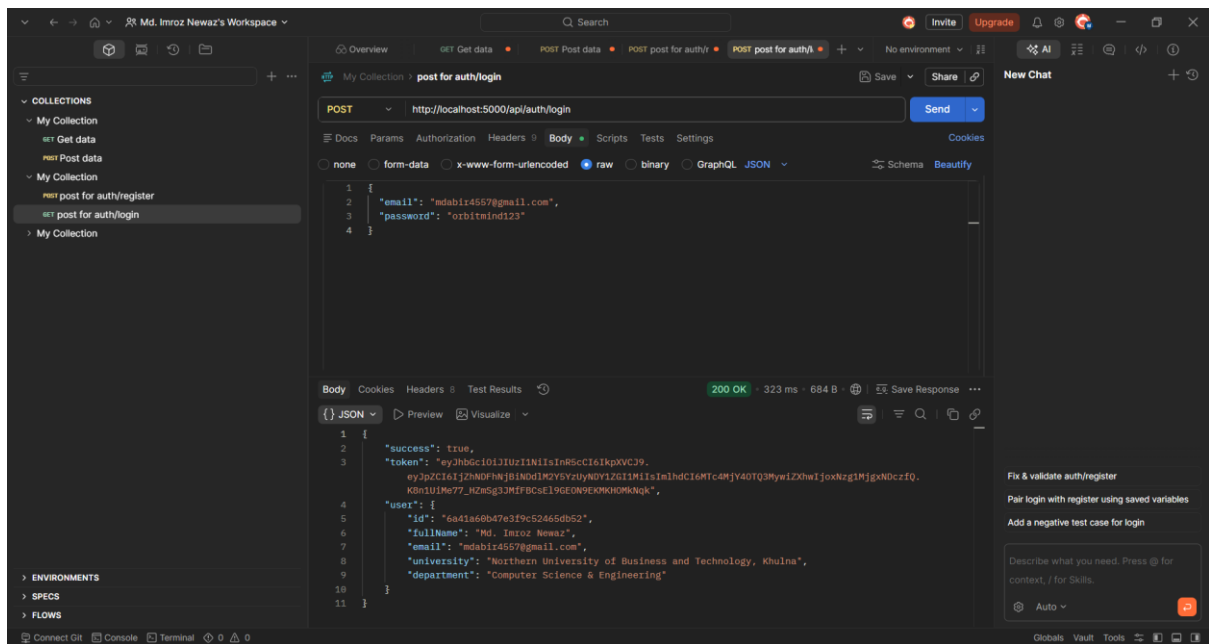


Figure 3.4.2: Successful API Testing of the Login Endpoint using Postman.

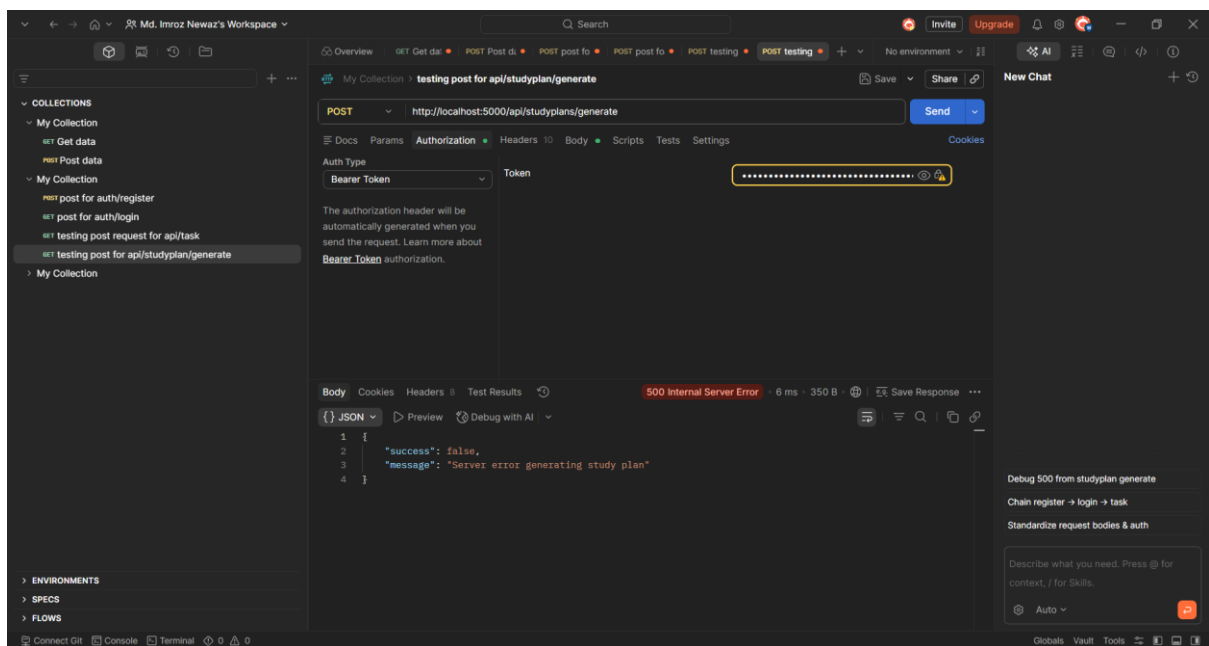


Figure 3.4.3: Failure API Testing of the Generate Endpoint using Postman.

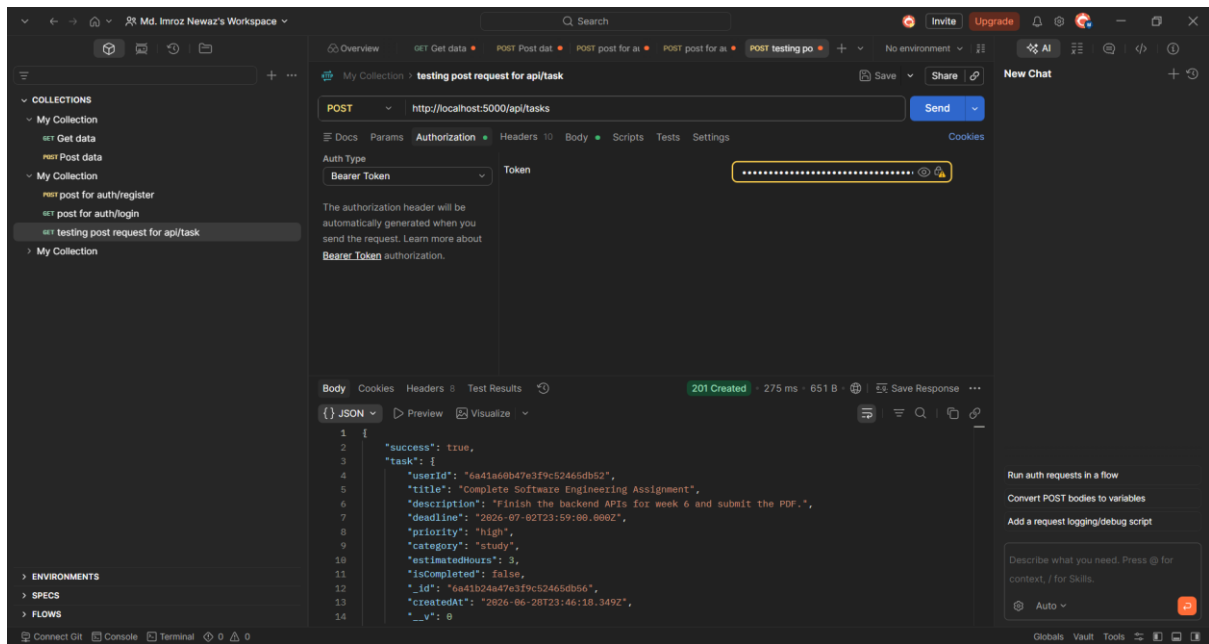


Figure 3.4.4: Successful API Testing of the Task Endpoint using Postman.

Implementation Details: These Postman interfaces confirm the successful, failure execution and routing of our core API endpoints. The HTTP status code and the returned JSON Web Token (JWT) verify that the backend is properly validating incoming data, processing the logic, and generating authentic sessions without crashing.

3.5 GitHub Repository

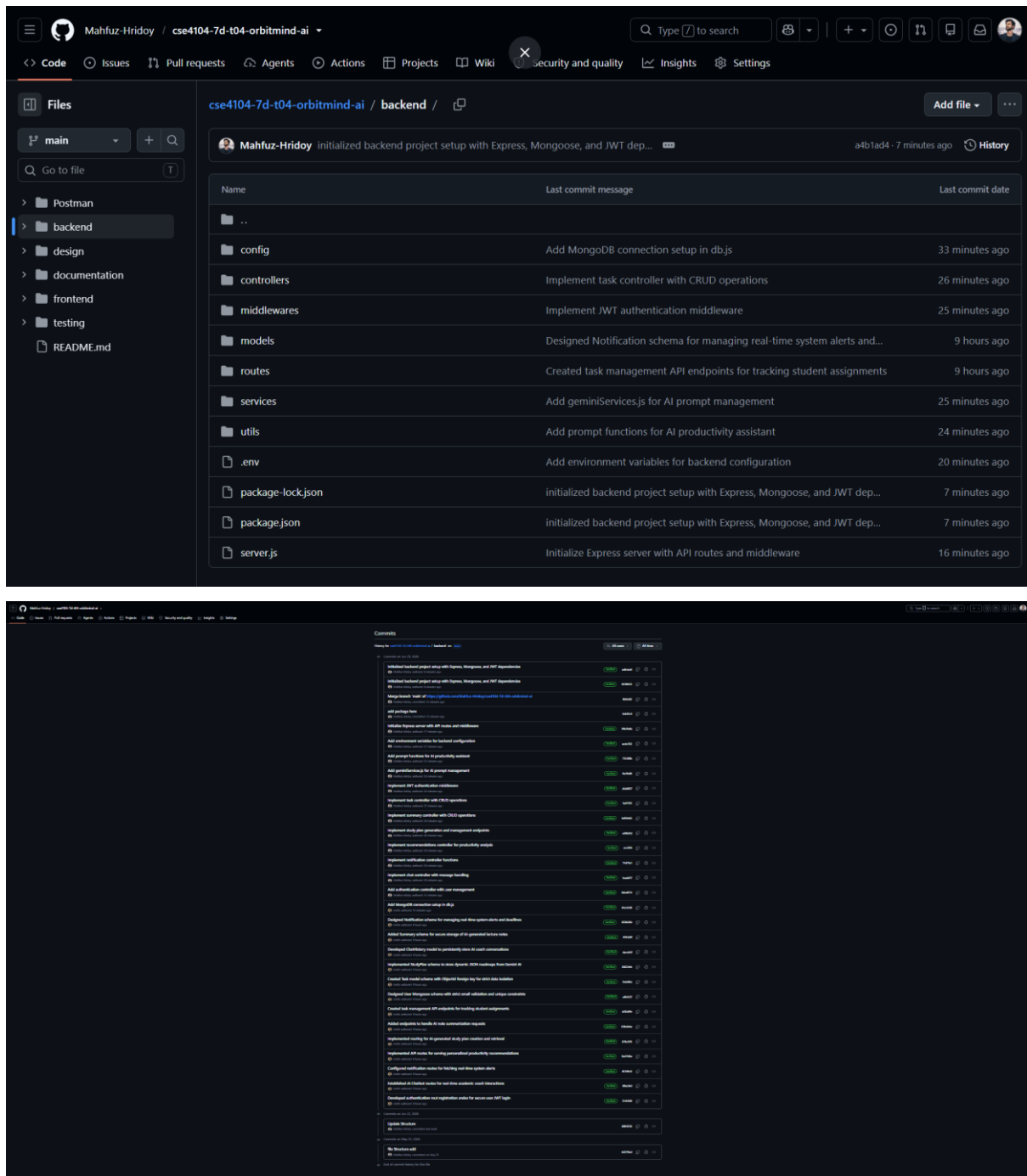


Figure 3.5: Maintained GitHub Repository showing organized branches and recent backend development commits.

Implementation Details: Our active GitHub repository reflects a professional and continuous development workflow. Meaningful commit messages document our progress throughout the

backend implementation phase, providing transparency to the instructor and facilitating seamless code collaboration among our team members.

3.5.1 GitHub Repository Link

Repository	Link
Official GitHub Repo	https://github.com/Mahfuz-Hridoy/cse4104-7d-t04-orbitmind-ai.git